Lab VI
(MUX)

Alex Rafael Soriano Severino
Mecatronica
ITLA
La Caleta, Santo Domingo
20240684@itla.edu.do

En resumen— En este laboratorio se diseñaron e implementaron las funciones lógicas Fa, Fb, Fc y Fd a partir de una tabla de verdad de cinco variables (A, B, C, D y E) utilizando multiplexores de 8x1 y 4x1. El propósito fue comprender el uso del MUX como dispositivo de síntesis lógica universal y su aplicación en la implementación de funciones booleanas complejas. Cada función fue diseñada primero con un MUX 8x1 y luego optimizada con un MUX 4x1. Posteriormente, se integraron las cuatro funciones en un MUX 4x1 general, logrando una única salida que, a través de un decodificador BCD, permitió mostrar el valor binario seleccionado en un display de 7 segmentos. Finalmente, se implementaron los ejercicios 3 y 4 de la planilla de laboratorio utilizando demultiplexores (DEMUX) en Multisim, verificando experimentalmente el funcionamiento correcto de la distribución de señales.

Abstract— In this laboratory, four Boolean functions (Fa, Fb, Fc, Fd) derived from a five-variable truth table were designed and implemented using 8x1 and 4x1 multiplexers. The objective was to explore the use of multiplexers as universal logic devices capable of synthesizing any Boolean function. Each function was built using an 8x1 MUX and then restructured with a 4x1 MUX. All outputs were combined into a single 4x1 multiplexer, whose binary output was decoded with a BCD-to-7-segment display. Additionally, two combinational problems were implemented using demultiplexers in Multisim, confirming the theoretical behavior through simulation.

***Palabras claves*— lógica combinacional, multiplexor, demultiplexor, Verilog, FPGA, BCD, Multisim, síntesis lógica.**

I. INTRODUCCION

El propósito de este laboratorio fue diseñar, analizar e implementar funciones lógicas a partir de multiplexores (MUX), aplicando los fundamentos de la lógica combinacional en la implementación física sobre una FPGA.
Se partió de una tabla de verdad con cinco variables de entrada (A, B, C, D, E), a partir de la cual se definieron las funciones Fa, Fb, Fc y Fd, que fueron implementadas primero con MUX 8x1 y luego con MUX 4x1, demostrando la equivalencia funcional entre ambas configuraciones.
Posteriormente, las cuatro funciones fueron integradas en un MUX 4x1 principal que selecciona cuál de las funciones se muestra en la salida. Esta salida fue conectada a un decodificador BCD y un display de 7 segmentos para mostrar el valor decimal correspondiente.
Además, se realizaron prácticas complementarias implementando DEMUX en Multisim, aplicando el principio inverso del MUX para distribuir una entrada hacia diferentes salidas.

II. MARCO TEÓRICO

A. Multiplexor (MUX)

El **multiplexor** es un circuito combinacional que selecciona una de entre varias entradas de datos y la envía a una salida común. Las líneas de selección determinan qué entrada es conectada. Matemáticamente, un MUX 8x1 puede representarse como: donde $D_i$son las entradas de datos y $m_i$los minterminos de las variables de selección. El MUX puede considerarse un **sintetizador** universal de funciones lógicas, pues cualquier función booleana puede implementarse configurando las entradas con los valores adecuados.

B. Demultiplexor (DEMUX)

El demultiplexor realiza la operación inversa del MUX: una sola entrada es dirigida hacia una de varias salidas posibles, dependiendo del valor de las líneas de selección. Se emplea en sistemas de distribución de datos, control de buses y decodificación de señales.

C. Implementación de funciones con MUX

Para implementar una función booleana con un MUX, se asignan las variables menos significativas a las líneas de selección y se colocan valores “1” o “0” en las entradas del MUX según los resultados de la tabla de verdad.

D. Decodificador BCD a Display

El decodificador BCD display traduce una entrada binaria en las señales necesarias para representar los dígitos 0–9 en un display de 7 segmentos. En este laboratorio se utilizó para visualizar la salida combinada del MUX principal en formato decimal.

E. *Diseño estructural en Verilog*

El enfoque estructural en Verilog consiste en instanciar módulos individuales (MUX 8x1, MUX 4x1, decodificador) dentro de un módulo principal, permitiendo una arquitectura jerárquica y modular. Este método facilita la implementación directa en FPGA y la simulación previa en software.

III. MONTAJE EXPERIMENTAL

Etapa 1: Diseño con MUX 8x1 y 4x1

1. A partir de la tabla de verdad de las variables A, B, C, D, E, se determinaron las salidas correspondientes de las funciones Fa, Fb, Fc y Fd.
2. Cada función fue implementada inicialmente con un MUX 8x1, utilizando las tres variables de menor peso como selectores.
3. Luego, las funciones fueron reconfiguradas con un MUX 4x1, ajustando las variables y entradas según la simplificación obtenida.
4. Las cuatro salidas (Fa, Fb, Fc, Fd) fueron conectadas a un MUX 4x1 final, generando una única salida controlada por los bits de selección S1 y S0.
5. La salida fue conectada a un decodificador BCD y posteriormente a un display de 7 segmentos, el cual mostró el valor decimal correspondiente al código binario seleccionado.

IV. FUNCIONAMIENTO DEL CIRCUITO:

El sistema opera con cinco entradas binarias (A, B, C, D, E). Cada función Fa, Fb, Fc y Fd representa una combinación lógica distinta derivada de la tabla de verdad proporcionada. Mediante las líneas de control del MUX 4x1 (S1 y S0), se selecciona qué función se muestra en la salida final.

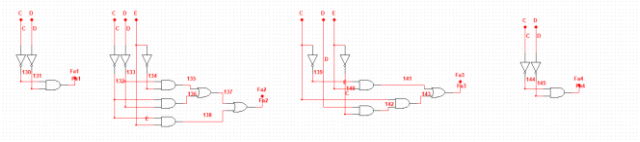
La tabla de selección general del MUX 4x1 fue:

S1	S0	Función
0	0	Fa
0	1	Fb
1	0	Fc
1	1	Fd

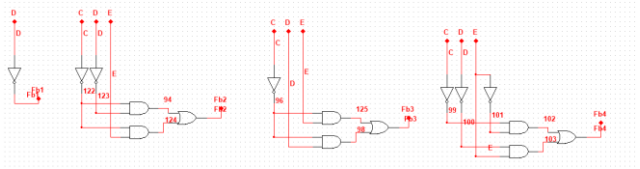
El valor de salida del MUX fue decodificado a BCD y visualizado en el display de 7 segmentos de la FPGA, confirmando el correcto funcionamiento de la selección lógica.

Diseño lógico (MULTISIM):

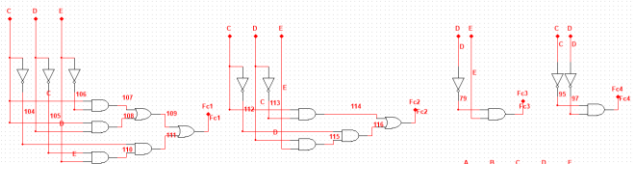
Función Fa:



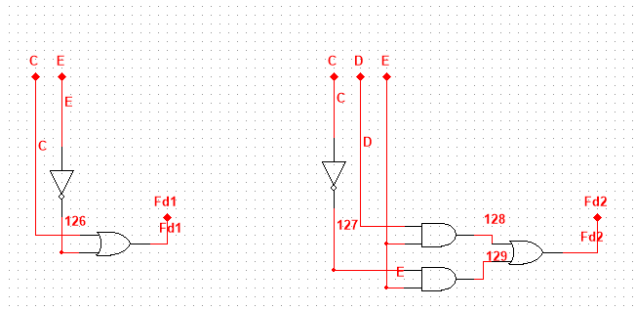
Función Fb:



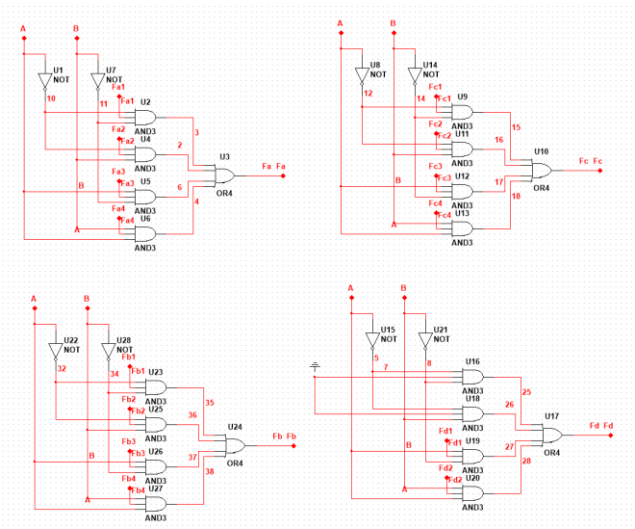
Función Fc:



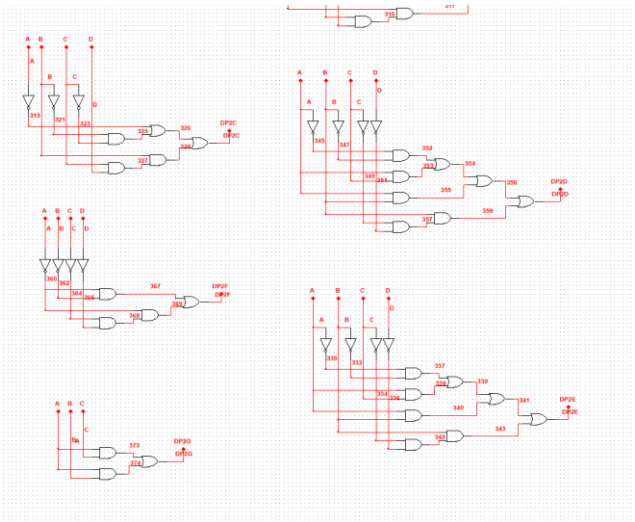
Función Fd:



Selección:



MUX:



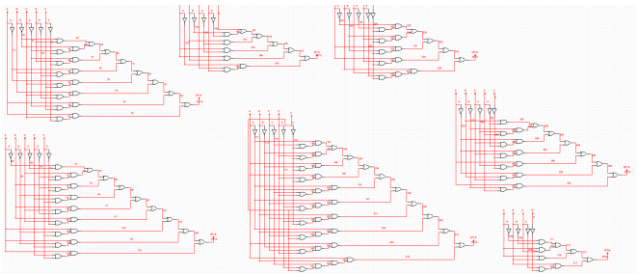
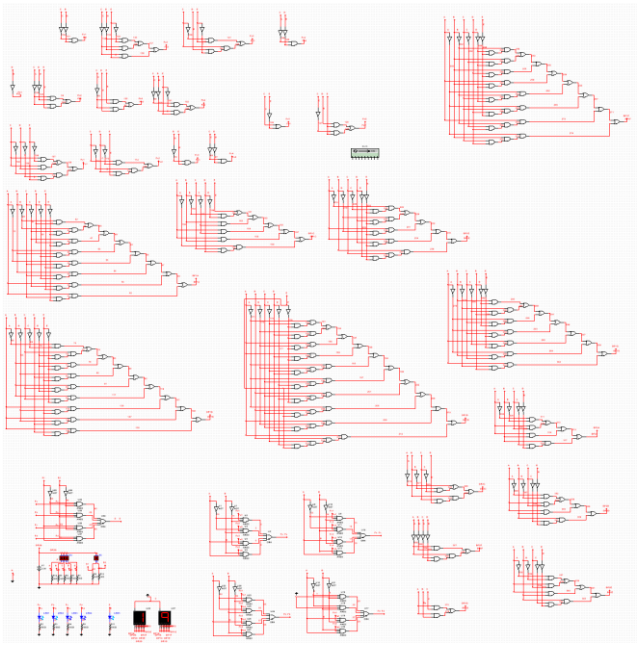
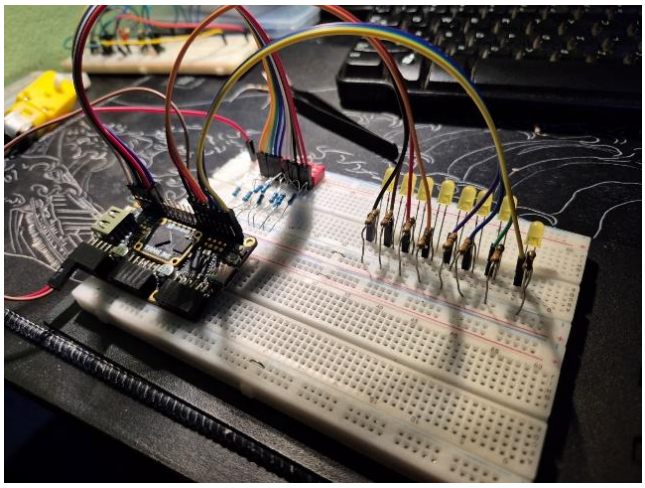


Diagrama del circuito completo:



Físico:



V. ANÁLISIS Y/O CÁLCULOS

Aquí calcularemos todos los datos solicitados en la práctica:

A. Tablas de Verdad Teórica

A partir de la tabla de verdad original (5 variables), se extrajeron los valores de salida para Fa, Fb, Fc y Fd, que sirvieron de base para las configuraciones de entradas del MUX.

	S0	S1	VEM						
	A	B	C	D	E	Fa	Fb	Fc	Fd
1	0	0	0	0	0	1	1	0	0
2	0	0	0	0	1	1	1	1	X
3	0	0	0	1	0	0	0	0	0
4	0	0	0	1	1	0	0	0	X
5	0	0	1	0	0	0	1	1	0
6	0	0	1	0	1	0	1	X	0
7	0	0	1	1	0	0	0	1	0
8	0	0	1	1	1	0	0	1	0
9	0	1	0	0	0	1	1	0	0
10	0	1	0	0	1	1	1	X	X
11	0	1	0	1	0	0	0	0	0
12	0	1	0	1	1	1	1	1	X
13	0	1	1	0	0	1	0	1	0
14	0	1	1	0	1	0	0	1	0
15	0	1	1	1	0	0	0	0	0
16	0	1	1	1	1	0	0	0	0
17	1	0	0	0	0	X	0	0	1
18	1	0	0	0	1	1	1	1	X
19	1	0	0	1	0	0	1	0	1
20	1	0	0	1	1	1	1	0	0
21	1	0	1	0	0	0	0	0	1
22	1	0	1	0	1	0	0	1	1
23	1	0	1	1	0	1	0	0	1
24	1	0	1	1	1	0	0	0	1
25	1	1	0	0	0	1	1	1	0
26	1	1	0	0	1	1	1	1	1
27	1	1	0	1	0	0	1	0	0
28	1	1	0	1	1	0	0	0	1
29	1	1	1	0	0	0	0	0	0
30	1	1	1	0	1	0	1	0	0
31	1	1	1	1	0	0	0	0	0
32	1	1	1	1	1	0	0	0	1

Estas son las tablas de verdad para el MUX:

	A	B	C	D	E	a	b	c	d	e	f	g	
0	0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	0	1	0	1	1	0	0	0	0	1
2	0	0	0	1	0	1	1	0	1	1	0	1	2
3	0	0	0	1	1	1	1	1	1	0	0	1	3
4	0	0	1	0	0	0	1	1	0	0	1	1	4
5	0	0	1	0	1	1	0	1	1	0	1	1	5
6	0	0	1	1	0	1	0	1	1	1	1	1	6
7	0	0	1	1	1	1	1	1	0	0	0	0	7
8	0	1	0	0	0	1	1	1	1	1	1	1	8
9	0	1	0	0	1	1	1	1	0	0	1	1	9
10	0	1	0	1	0	1	1	1	1	1	1	0	0
11	0	1	0	1	1	0	1	1	0	0	0	0	1
12	0	1	1	0	0	1	1	0	1	1	0	1	2
13	0	1	1	0	1	1	1	1	1	0	0	1	3
14	0	1	1	1	0	0	1	1	0	0	1	1	4
15	0	1	1	1	1	1	0	1	1	0	1	1	5
16	1	0	0	0	0	1	0	1	1	1	1	1	6
17	1	0	0	0	1	1	1	1	0	0	0	0	7
18	1	0	0	1	0	1	1	1	1	1	1	1	8
19	1	0	0	1	1	1	1	1	0	0	1	1	9
20	1	0	1	0	0	1	1	1	1	1	1	0	0
21	1	0	1	0	1	0	1	1	0	0	0	0	1
22	1	0	1	1	0	1	1	0	1	1	0	1	2
23	1	0	1	1	1	1	1	1	1	0	0	1	3
24	1	1	0	0	0	0	1	1	0	0	1	1	4
25	1	1	0	0	1	1	0	1	1	0	1	1	5
26	1	1	0	1	0	1	0	1	1	1	1	1	6
27	1	1	0	1	1	1	1	1	0	0	0	0	7
28	1	1	1	0	0	1	1	1	1	1	1	1	8
29	1	1	1	0	1	1	1	1	0	0	1	1	9
30	1	1	1	1	0	1	1	1	1	1	1	0	0
31	1	1	1	1	1	0	1	1	0	0	0	0	1

	A	B	C	D	E	a	b	c	d	e	f	g	
0	0	0	0	0	0	1	1	1	1	1	1	0	0
1	0	0	0	0	1	1	1	1	1	1	1	0	0
2	0	0	0	1	0	1	1	1	1	1	1	0	0
3	0	0	0	1	1	1	1	1	1	1	1	0	0
4	0	0	1	0	0	1	1	1	1	1	1	0	0
5	0	0	1	0	1	1	1	1	1	1	1	0	0
6	0	0	1	1	0	1	1	1	1	1	1	0	0
7	0	0	1	1	1	1	1	1	1	1	1	0	0
8	0	1	0	0	0	1	1	1	1	1	1	0	0
9	0	1	0	0	1	1	1	1	1	1	1	0	0
10	0	1	0	1	0	0	1	1	0	0	0	0	1
11	0	1	0	1	1	0	1	1	0	0	0	0	1
12	0	1	1	0	0	0	1	1	0	0	0	0	1
13	0	1	1	0	1	0	1	1	0	0	0	0	1
14	0	1	1	1	0	0	1	1	0	0	0	0	1
15	0	1	1	1	1	0	1	1	0	0	0	0	1
16	1	0	0	0	0	0	1	1	0	0	0	0	1
17	1	0	0	0	1	0	1	1	0	0	0	0	1
18	1	0	0	1	0	0	1	1	0	0	0	0	1
19	1	0	0	1	1	0	1	1	0	0	0	0	1
20	1	0	1	0	0	1	1	0	1	1	0	1	2
21	1	0	1	0	1	1	1	0	1	1	0	1	2
22	1	0	1	1	0	1	1	0	1	1	0	1	2
23	1	0	1	1	1	1	1	0	1	1	0	1	2
24	1	1	0	0	0	1	1	0	1	1	0	1	2
25	1	1	0	0	1	1	1	0	1	1	0	1	2
26	1	1	0	1	0	1	1	0	1	1	0	1	2
27	1	1	0	1	1	1	1	0	1	1	0	1	2
28	1	1	1	0	0	1	1	0	1	1	0	1	2
29	1	1	1	0	1	1	1	0	1	1	0	1	2
30	1	1	1	1	0	1	1	1	1	0	0	1	3
31	1	1	1	1	1	1	1	1	1	0	0	1	3

B. Mapas de Karnaugh

Mux 1

Salida (Fa)				
C/DE	00	01	11	10
0	1	1	0	0
1	0	0	0	0

SoP
C'D'

Salida (Fa)				
C/DE	00	01	11	10
0	1	1	1	0
1	1	0	0	0

SoP
D'E' + C'D' + C'E

Salida (Fa)				
C/DE	00	01	11	10
0	X	1	1	0
1	0	0	0	1

SoP
C'E + CDE'

Salida (Fa)				
C/DE	00	01	11	10
0	1	1	0	0
1	0	0	0	0

SoP
C'D'

Mux 2

Salida (Fb)				
C/DE	00	01	11	10
0	1	1	0	0
1	1	1	0	0

SoP
D'

Salida (Fb)				
C/DE	00	01	11	10
0	1	1	1	0
1	0	0	0	0

SoP
C'D' + C'E

Salida (Fb)				
C/DE	00	01	11	10
0	0	1	1	1
1	0	0	0	0

SoP
C'E + C'D

Salida (Fb)				
C/DE	00	01	11	10
0	1	1	0	1
1	0	1	0	0

SoP
C'E' + D'E

Mux 3

Salida (Fc)				
C/DE	00	01	11	10
0	0	1	0	0
1	1	X	1	1

SoP
CE' + CD + C'D'E

Salida (Fc)				
C/DE	00	01	11	10
0	0	X	1	0
1	1	1	0	0

SoP
CD' + C'DE

Salida (Fc)				
C/DE	00	01	11	10
0	0	1	0	0
1	0	1	0	0

SoP
D'E

Salida (Fc)				
C/DE	00	01	11	10
0	1	1	0	0
1	0	0	0	0

SoP
C'D'

Mux 4

Salida (Fd)				
C/DE	00	01	11	10
0	0	X	X	0
1	0	0	0	0

SoP
0

Salida (Fd)				
C/DE	00	01	11	10
0	0	X	X	0
1	0	0	0	0

SoP
0

Salida (Fd)				
C/DE	00	01	11	10
0	1	X	0	1
1	1	1	1	1

SoP
C + E'

Salida (Fd)				
C/DE	00	01	11	10
0	0	1	1	0
1	0	0	1	0

SoP
DE + C'E

C. Funciones Lógicas del MUX

SoP (a)
B'D + B'C'E' + C'DE' + A'BD' + A'CE + BCD' + AC'E + ACE'

SoP (b)
A'C' + CD'E' + BD'E' + B'DE + BCD' + BCE' + AB'E + AB'D + ADE

SoP (c)
E + B'D' + BC' + AC' + AB + A'CD

SoP (d)
A'C'E' + B'DE' + AB'E' + ADE' + A'B'C'D + A'CD'E + BCD'E' + A'BCE + AB'CD + ABC'D'E

SoP (e)
A'C'E' + B'DE' + AB'E' + ADE' + BCD'E'

SoP (f)
B'D'E' + BC'D' + BDE' + ABD' + A'B'CD' + A'CDE' + AB'C'D + A'BCD

SoP (g)
BD' + B'C'D + A'CD' + A'CE' + AC'E' + A'BC + AB'D

SoP (a)
A'B' + AC + AB + BC'D'

SoP (b)
1

SoP (c)
A' + B'C' + BCD

SoP (d)
A'B' + AC + AB + BC'D'

SoP (e)
A'B' + B'C + BC'D' + ABC' + ABD'

SoP (f)
A'B' + A'C'D'

SoP (g)
AC + AB

D. Programación en Gowin con verilog

Archivo MAIN.v:

```
//Module MAIN
module MAIN(
    // Inputs and outputs declaration
    input A,B,C,D,E,S0,S1,
    output O
);

    wire A1,A2,A3,A4,B1,B2,B3,B4,C1,C2,C3,C4,D1,D2;
    wire FA,FB,FC,FD;

    //INSTANCES FA
    FA1 FA_1 (
        .C(C),
        .D(D),

        .FAS1(A1)
    );

    FA2 FA_2 (
        .C(C),
        .D(D),
        .E(E),

        .FAS2(A2)
    );
```

```
    FA3 FA_3 (
        .C(C),
        .D(D),
        .E(E),

        .FAS3(A3)
    );

    FA4 FA_4 (
        .C(C),
        .D(D),

        .FAS4(A4)
    );

    MUX MUXFA (
        .A(A),
        .B(B),
        .F1(A1),
        .F2(A2),
        .F3(A3),
        .F4(A4),

        .F(FA)
    );

    //INSTANCES FB
    FB1 FB_1 (
        .D(D),

        .FBS1(B1)
    );

    FB2 FB_2 (
        .C(C),
        .D(D),
        .E(E),

        .FBS2(B2)
    );

    FB3 FB_3 (
        .C(C),
        .D(D),
        .E(E),

        .FBS3(B3)
    );

    FB4 FB_4 (
        .C(C),
        .D(D),
        .E(E),

        .FBS4(B4)
    );

    MUX MUXFB (
        .A(A),
        .B(B),
        .F1(B1),
        .F2(B2),
        .F3(B3),
        .F4(B4),

        .F(FB)
    );
```

Fa:

```
//Module Fa1
module FA1(
    // Inputs and outputs declaration
    input C,D,
    output FAS1
);

    wire  C1,C2;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    and AND1(FAS1,C1,C2);
endmodule

//Module Fa2
module FA2(
    // Inputs and outputs declaration
    input C,D,E,
    output FAS2
);

    wire  C1,C2,C3,C4,C5,C6,C7;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    not N3(C3,E);
    and AND1(C4,C2,C3);
    and AND2(C5,C1,C2);
    and AND3(C6,C1,E);
    or OR1(C7,C4,C5);
    or OR2(FAS2,C6,C7);
endmodule

//Module Fa3
module FA3(
    // Inputs and outputs declaration
    input C,D,E,
    output FAS3
);

    wire  C1,C2,C3,C4,C5,C6,C7;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    not N3(C3,E);
    and AND1(C4,C2,C3);
    and AND2(C5,C1,C2);
    and AND3(C6,C1,E);
    or OR1(C7,C4,C5);
    or OR2(FAS3,C6,C7);
endmodule

//Module Fa4
module FA4(
    // Inputs and outputs declaration
    input C,D,
    output FAS4
);

    wire  C1,C2;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    and AND1(FAS4,C1,C2);
```

Fb:

```
//Module Fb1
module FB1(
    // Inputs and outputs declaration
    input D,
    output FBS1
);

    //declarations
    not N1(FBS1,D);
endmodule

//Module Fb2
module FB2(
    // Inputs and outputs declaration
    input C,D,E,
    output FBS2
);

    wire  C1,C2,C3,C4;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    and AND1(C3,C1,C2);
    and AND2(C4,C1,E);
    or OR1(FBS2,C3,C4);
endmodule

//Module Fb3
module FB3(
    // Inputs and outputs declaration
    input C,D,E,
    output FBS3
);

    wire  C1,C2,C3,C4;

    //declarations
    not N1(C1,C);
    and AND1(C2,C1,E);
    and AND2(C3,C1,D);
    or OR1(FBS3,C2,C3);
endmodule

//Module Fb4
module FB4(
    // Inputs and outputs declaration
    input C,D,E,
    output FBS4
);

    wire  C1,C2,C3,C4;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    not N3(C3,E);
    and AND1(C4,C1,C3);
    and AND2(C5,C2,E);
    or OR1(FBS4,C4,C5);
endmodule
```

Fc:

```
module FC1(
    // Inputs and outputs declaration
    input C,D,E,
    output FCS1
);

    wire  C1,C2,C3,C4,C5,C6,C7,C8;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    not N3(C3,E);
    and AND1(C4,C,C3);
    and AND2(C5,C,D);
    and AND3(C6,C2,E);
    and AND4(C7,C1,C6);
    or OR1(C8,C4,C5);
    or OR2(FCS1,C8,C7);
endmodule

//Module Fc2
module FC2(
    // Inputs and outputs declaration
    input C,D,E,
    output FCS2
);

    wire  C1,C2,C3,C4;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    and AND1(C3,C,C2);
    and AND2(C4,D,E);
    and AND3(C5,C1,C4);
    or OR1(FCS2,C3,C5);
endmodule

//Module Fc3
module FC3(
    // Inputs and outputs declaration
    input D,E,
    output FCS3
);

    wire  C1;

    //declarations
    not N1(C1,D);
    and AND1(FCS3,C1,E);
endmodule

//Module Fc4
module FC4(
    // Inputs and outputs declaration
    input C,D,
    output FCS4
);

    wire  C1,C2,C3,C4;

    //declarations
    not N1(C1,C);
    not N2(C2,D);
    and AND1(FCS4,C1,C2);
endmodule
```

Fd:

```
//Module Fd1
module FD1(
    // Inputs and outputs declaration
    input C,E,
    output FDS1
);

    wire  C1;

    //declarations
    not N1(C1,E);
    or OR1(FDS1,C,C1);
endmodule

//Module Fd2
module FD2(
    // Inputs and outputs declaration
    input C,D,E,
    output FDS2
);

    wire  C1,C2,C3;

    //declarations
    not N1(C1,C);
    and AND1(C2,D,E);
    and AND2(C3,C1,E);
    or OR1(FDS2,C2,C3);
endmodule
```

MUX:

```
module MUX(
    // Inputs and outputs declaration
    input A,B,F1,F2,F3,F4,
    output F
);

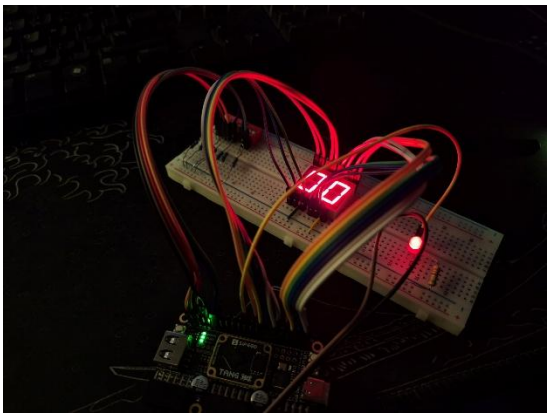
    wire  C1,C2,C3,C4,C5,C6;

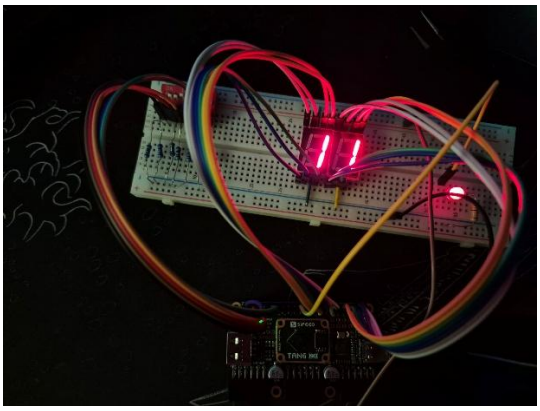
    //declarations
    not N1(C1,A);
    not N2(C2,B);
    and AND1(C3,C1,F1,C2);
    and AND2(C4,C1,F2,B);
    and AND3(C5,A,F3,C2);
    and AND4(C6,B,F4,A);
    or OR1(F,C3,C4,C5,C6);
endmodule
```

LINK AL VIDEO:

https://itlaedudo-my.sharepoint.com/:v:/g/personal/20240684_itla_edu_do/EYrAjwOzVhlHhnHx_ngcEpIBp6bel9_lYz7QOAMwB86wbA?e=eiTdYv

VI. FOTOS DEL CIRCUITO EN FUNCIONAMIENTO





VII. CONCLUSION

El laboratorio permitió comprender de forma práctica la función del multiplexor como herramienta universal de síntesis lógica, demostrando su capacidad para implementar cualquier función booleana.

Las funciones Fa, Fb, Fc y Fd se implementaron correctamente tanto con MUX 8x1 como con MUX 4x1, evidenciando la flexibilidad del diseño combinacional y la relación entre complejidad y cantidad de entradas.

El montaje en FPGA mostró un funcionamiento estable y coherente con la simulación en Multisim, confirmando la correcta codificación en Verilog estructural y la integración del decodificador BCD–display.

Asimismo, la implementación de los DEMUX permitió afianzar el concepto inverso del MUX, completando la comprensión teórica y práctica de ambos dispositivos.

En conclusión, el laboratorio reforzó los conocimientos de lógica combinacional, diseño modular y síntesis digital, consolidando la capacidad de aplicar estos principios en implementaciones físicas de sistemas digitales.

VIII. REFERENCIAS

https://en.wikipedia.org/wiki/Digital_electronics
https://naylampmechatronics.com/blog/39_resistencias-pull-up-y-pull-down.html
https://es.wikipedia.org/wiki/Sistema_de_numeraci%C3%B3n
https://es.wikipedia.org/wiki/C%C3%B3digo_binario
https://es.slideshare.net/slideshow/sistemas-de-codigos-en-la-electronica-digital/158426000